

Feature Scaling

Aim

To perform data preprocessing and feature scaling in Python using SimpleImputer, OneHotEncoder, MinMaxScaler, and StandardScaler to prepare the dataset for machine learning models.

Procedure

1. Import the necessary libraries such as pandas, numpy, and preprocessing tools from sklearn.
2. Load the dataset into a pandas DataFrame for preprocessing.
3. Identify missing values in the dataset using `isnull().sum()` to determine where imputation is required.
4. Use SimpleImputer to fill missing numerical or categorical values with appropriate strategies such as mean, median, or most frequent value.
5. Separate categorical and numerical columns for independent processing.
6. Apply OneHotEncoder to convert categorical variables into binary (0/1) encoded columns suitable for machine learning algorithms.
7. Use MinMaxScaler to normalize numerical features within a fixed range, usually between 0 and 1.
8. Apply StandardScaler to standardize features so that they have a mean of 0 and a standard deviation of 1.
9. Combine the processed numerical and encoded categorical features into a single dataset.
10. Verify the transformed dataset to ensure all missing values are handled and all features are properly scaled.

```
import numpy as np
import pandas as pd
fp="/pre_process_datasample - pre_process_datasample.csv"
df=pd.read_csv(fp)
df.head()
```

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	NaN	Yes

```
df.isnull().sum()
```

```
0
Country    1
Age        1
Salary     1
Purchased  0
dtype: int64
```

```
df.Country.fillna(df['Country'].mode()[0],inplace=True)
df
```

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	NaN	Yes
5	France	35.0	58000.0	Yes
6	Spain	NaN	52000.0	No
7	France	48.0	79000.0	Yes
8	Germany	50.0	83000.0	No
9	France	37.0	67000.0	Yes

```
labels=df.iloc[:, :-1].values
features=df.iloc[:, :-1].values
labels
```

```
array([[ 'France', 44.0, 72000.0],
       [ 'Spain', 27.0, 48000.0],
       [ 'Germany', 30.0, 54000.0],
       [ 'Spain', 38.0, 61000.0],
       [ 'Germany', 40.0, nan],
       [ 'France', 35.0, 58000.0],
       [ 'Spain', nan, 52000.0],
       [ 'France', 48.0, 79000.0],
       [ 'Germany', 50.0, 83000.0],
       [ 'France', 37.0, 67000.0]], dtype=object)
```

```
from sklearn.impute import SimpleImputer
age=SimpleImputer(strategy='mean',missing_values=np.nan)
salary=SimpleImputer(strategy='mean',missing_values=np.nan)
features[:,1:2]=age.fit_transform(features[:,1:2])
features[:,2:3]=salary.fit_transform(features[:,2:3])
features
```

```
array([[ 'France', 44.0, 72000.0],
       [ 'Spain', 27.0, 48000.0],
       [ 'Germany', 30.0, 54000.0],
       [ 'Spain', 38.0, 61000.0],
```

```
[ 'Germany', 40.0, 63777.777777777778],
[ 'France', 35.0, 58000.0],
[ 'Spain', 38.77777777777778, 52000.0],
[ 'France', 48.0, 79000.0],
[ 'Germany', 50.0, 83000.0],
[ 'France', 37.0, 67000.0]], dtype=object)
```

```
from sklearn.preprocessing import OneHotEncoder
oh=OneHotEncoder(sparse_output=False)
Country=oh.fit_transform(features[:,[0]])
Country
```

```
array([[1., 0., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [1., 0., 0.],
       [0., 0., 1.],
       [1., 0., 0.],
       [0., 1., 0.],
       [1., 0., 0.]])
```

```
final=np.concatenate((Country,features[:,[1,2]]),axis=1)
```

```
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
sc.fit(final)
feat_standard_scaler=sc.transform(final)
feat_standard_scaler
```

```
array([[ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
         7.58874362e-01,  7.49473254e-01],
       [-8.16496581e-01, -6.54653671e-01,  1.52752523e+00,
        -1.71150388e+00, -1.43817841e+00],
       [-8.16496581e-01,  1.52752523e+00, -6.54653671e-01,
        -1.27555478e+00, -8.91265492e-01],
       [-8.16496581e-01, -6.54653671e-01,  1.52752523e+00,
        -1.13023841e-01, -2.53200424e-01],
       [-8.16496581e-01,  1.52752523e+00, -6.54653671e-01,
         1.77608893e-01,  6.63219199e-16],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
        -5.48972942e-01, -5.26656882e-01],
       [-8.16496581e-01, -6.54653671e-01,  1.52752523e+00,
         0.00000000e+00, -1.07356980e+00],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
         1.34013983e+00,  1.38753832e+00],
       [-8.16496581e-01,  1.52752523e+00, -6.54653671e-01,
         1.63077256e+00,  1.75214693e+00],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
        -2.58340208e-01,  2.93712492e-01]])
```

```
from sklearn.preprocessing import MinMaxScaler
mm=MinMaxScaler()
mm.fit(final)
feat_min_max_scaler=mm.transform(final)
feat_min_max_scaler
```

```
array([[1.         , 0.         , 0.         , 0.73913043, 0.68571429],
       [0.         , 0.         , 1.         , 0.         , 0.         ],
       [0.         , 1.         , 0.         , 0.13043478, 0.17142857],
       [0.         , 0.         , 1.         , 0.47826087, 0.37142857],
       [0.         , 1.         , 0.         , 0.56521739, 0.45079365],
       [1.         , 0.         , 0.         , 0.34782609, 0.28571429],
       [0.         , 0.         , 1.         , 0.51207729, 0.11428571],
       [1.         , 0.         , 0.         , 0.91304348, 0.88571429],
       [0.         , 1.         , 0.         , 1.         , 1.         ],
       [1.         , 0.         , 0.         , 0.43478261, 0.54285714]])
```

Result:

Hence, a Python program to preprocess data using SimpleImputer, OneHotEncoder, MinMaxScaler, and StandardScaler has been implemented successfully, producing a clean and scaled dataset ready for machine learning model training.